Swin3D: A Pretrained Transformer Backbone for 3D Indoor Scene Understanding

Yu-Qi Yang[1*] Yu-Xiao Guo[2] Jian-Yu Xiong[1*] Yang Liu[2†]
Hao Pan[2] Peng-Shuai Wang[3] Xin Tong[2] Baining Guo[2]
[1]Tsinghua University [2]Microsoft Research Asia [3]Peking University
{t-yuqyan, yuxgu, v-jixiong, yangliu}@microsoft.com
haopan@microsoft.com wangps@hotmail.com {xtong,bainguo}@microsoft.com

Abstract

The use of pretrained backbones with fine-tuning has been successful for 2D vision and natural language processing tasks, showing advantages over task-specific networks. In this work, we introduce a pretrained 3D backbone, called SWIN3D*, for 3D indoor scene understanding. We design a 3D Swin transformer as our backbone network, which enables efficient self-attention on sparse voxels with linear memory complexity, making the backbone scalable to large models and datasets. We also introduce a generalized contextual relative positional embedding scheme to capture various irregularities of point signals for improved network performance. We pretrained a large* SWIN3D *model on a synthetic Structured3D dataset, which is an order of magnitude larger than the ScanNet dataset. Our model pretrained on the synthetic dataset not only generalizes well to downstream segmentation and detection on real 3D point datasets, but also outperforms state-of-the-art methods on downstream tasks with +2.3 mIoU and +2.2 mIoU on S3DIS Area5 and 6-fold semantic segmentation, +1.8 mIoU on ScanNet segmentation (val), +1.9 mAP@0.5 on ScanNet detection, and +8.1 mAP@0.5 on S3DIS detection. A series of extensive ablation studies further validate the scalability, generality, and superior performance enabled by our approach.*

1. Introduction

A paradigm shift has been seen in the fields of Natural Language Processing (NLP) and 2D vision, where the use of large pre-trained backbones has been highly successful [15, 2, 12, 4, 33, 32]. This approach has the advantage of being able to generalize to various tasks, while also reducing the amount of network design and training needed, as well as the amount of labeled data required for various vision tasks. This involves pre-training a backbone network with a simple design on a large dataset, and then fine-tuning it for different downstream tasks. However, the development of generic and scalable pretrained 3D backbones for point cloud understanding is still in its early stages, and existing pretrained 3D backbones are not as effective as the state-of-the-art non-pretrained approaches for many 3D vision tasks.

This paper seeks to investigate the scalability and generality of a 3D pretrained model, without the need for a complex network design. We introduce a pretrained 3D backbone, SWIN3D, for 3D indoor scene understanding tasks. Our method uses sparse voxels to represent the 3D point cloud of an input 3D scene and adapts the network design of Swin Transformer [33], which was originally designed for regular 2D images, to unorganized 3D points as the 3D backbone. We identify two key issues that prevent the naïve 3D extension of Swin Transformer from exploring large models and achieving high performance: *the high memory complexity* and *the ignorance of signal irregularity*. To address these issues, we develop a novel 3D self-attention operator to compute the self-attentions of sparse voxels within each local window. This reduces the memory cost of self-attention from quadratic to linear with respect to the number of sparse voxels within a window and computes efficiently without sacrificing self-attention accuracy. Additionally, it enhances self-attention by capturing various signal irregularities by generalizing *contextual relative positional embedding* [52, 28] to point signals.

Our novel SWIN3D backbone design allows us to scale up the backbone model and utilize the large amount of data for pretraining. To demonstrate this, we pretrained a large SWIN3D model with 60 M parameters on a 3D semantic segmentation task using a large synthetic 3D dataset: Structured3D [65]. This dataset is ten times larger than the ScanNet dataset [11] that contains 21 K rooms. After pretraining, we combined the pretrained SWIN3D backbone with task-specific back-end decoders and fine-tuned the models for

*Interns at Microsoft Research Asia. †Contact person.

various 3D indoor scene understanding tasks.

We evaluated the performance of our method on both 3D detection and semantic segmentation tasks on real data including ScanNet and S3DIS datasets. Experimental results show that our SWIN3D pretrained on the synthetic dataset exhibits good generality and outperforms all existing state-of-the-art methods with +8.1 mAP@0.5 on S3DIS detection, +2.2 mIoU on 6-fold S3DIS segmentation [1], +1.9 mAP@0.5 on ScanNet detection and +1.8 mIoU on ScanNet segmentation (validation set) respectively. We carefully analyzed the contributions of different factors (*e.g.*, model size, data size, pretraining) to the performance of our model. Our studies show that our pretrained backbones with fine-tuning are superior to the same models trained from scratch and outperform other existing models pretrained with the same large data significantly.

The large backbone model and the large amount of 3D data used for pretraining enabled by our backbone design are critical to the superior performance of our method in the downstream tasks. We believe that our work illustrates the great potential of a unified pretrained backbone to various 3D understanding tasks. To facilitate and inspire future research along this path, we have released our code and trained models at `https://github.com/microsoft/Swin3D`.

2. Related Work

Vision transformers Transformers based on the attention mechanism have been used successfully in computer vision and have achieved great results in many 2D vision tasks, such as image classification, semantic segmentation, and object detection (*cf.* the comprehensive surveys [26, 20, 17]). The plain vision transformer [15] computes global self-attention over the entire image, thus providing long-range attention between image patches, but this leads to high memory and computational costs due to the quadratic complexity of self-attention. To address this issue, local-window self-attention over small non-overlapping patches [21, 33] was introduced. Various techniques have been proposed to improve the long-range attention of window-based self-attention, such as using window hierarchy [33], constructing non-local self-attention patterns [14, 43, 61, 53, 58, 29, 50, 10], and expanding the receptive field via convolution [6, 60]. Most vision transformers are pretrained with large-scale image datasets and serve as generic vision backbones for multiple purposes.

3D transformers for point cloud understanding Transformers have been quickly adapted to 3D [27]. Guo *et al*. [16] used global attention on points and achieved good results in object classification and shape segmentation. Zhao *et al*. [64] introduced local attention on point clouds, which reduced memory and computational complexity and enabled the point transformer to be used for point clouds at the scene level. Wu *et al*. [55] further improved the point transformer by using grouped vector attention and partition-based pooling. Fast Point Transformer [36] employed voxel hashing and a lightweight self-attention layer to enhance network efficiency. Stratified Transformer [28] adapted the Swin Transformer design for 3D point clouds and proposed the stratified strategy to expand the reception field and used contextual relative positional encoding [52] to strengthen self-attention with position information. In 3D vision, most 3D transformers that have been created have been tailored to particular tasks, and simply training them on a large amount of data does not result in better performance, as we evaluated (Section 6).

Pretrained 3D backbones Self-supervised learning strategies have been applied to 3D backbone pretraining. PointContrast [57] used point-level losses to pretrain a sparse-convolution-based 3D U-Net. MID-Net [48] pretrained an Octree-based HRNet with multiresolution contrastive losses. Hou *et al*. [23] improved the efficacy of PointContrast by taking advantage of spatial information. DepthContrast [63] employed depth maps to enhance contrastive learning. Masked-signal-modeling-based transformer models, such as BEiT [2] and the masked autoencoder [22], have been used for 3D pretraining [31, 62, 59, 35]. Wu Wu *et al*. [56] intergrated the masked point modeling strrategy with contrastive learning to boost backbone pretraining. Recently, pretrained image or CLIP models have been utilized for 3D learning [51, 13, 24, 25], forming a new type of pretrained 3D backbones. ShapeNet [5] and ScannNet [11] are the main 3D data sources for the above pre-training work. Despite the rapid development of 3D pretraining, its performance on 3D indoor scene understanding is still inferior to that of the state-of-the-art non-pretrained approaches.

3. Architecture overview

3.1. Naive 3D extension of Swin Transformer

Hierarchical window-based transformers, such as Swin Transformer [33], are widely used in generic vision due to their high efficiency, multiscale feature learning, scalability, and improved performance compared to 2D CNN backbones. It is thus a logical step to extend Swin Transformer-like architectures for 3D point cloud learning. The implementation of window attention mechanism in 3D appears to be straightforward — partition the input 3D point cloud into non-overlapping 3D windows and compute self-attention on nonempty-voxel features within regular and shifted windows. However, as Lai *et al*. [28] observed in their ablation study, this naive extension does not lead to superior performance. We have identified two key issues that explain this uninspiring performance: *memory complexity* and *signal irregularity*.

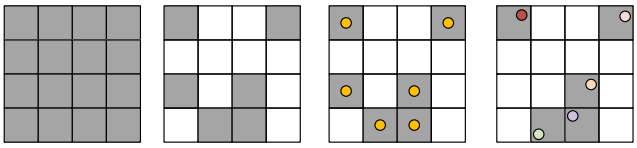

Figure 1. A 2D illustration of sparse points in a 4×4 window. **(a)** A fully-occupied window. **(b)** A sparsely-occuppied window, with white cells being empty. **(c)** Regularly-distributed sparse points in a window. **(d)** The sparse points are irregularly distributed in the window, and different circle colors indicate the varying point-wise signal, such as the RGB color. For simplicity, only one point is drawn on non-empty cells.

Memory complexity The quadratic complexity of self-attention leads to a high memory cost in 3D. For a 3D window with size $M \times M \times M$, the average number of non-empty voxels within the window is approximately $\mathcal{O}(M^2)$, thus the memory cost of executing vanilla self-attention within the window is about $\mathcal{O}(M^4)$. For a 3D scene, the number of windows N_w could be considerable, depending on the size of the 3D scenes. Therefore, the total memory cost $\mathcal{O}(N_w M^4)$ in 3D could be much higher than its 2D counterpart, where the image size is usually low. This memory issue prevents the use of large windows and more Swin layers, making it difficult to design large models that can benefit from large data.

Signal irregularity The locations of 3D points can be highly irregular: points can be found anywhere within their occupied voxel, while for 2D visual transformers, image pixels are regularly distributed at grid cell centers. Additionally, since points are usually equipped with other raw signals, such as RGB colors, if we consider both point positions and other pointwise signals as voxel signals, the signal irregularity, *i.e.*, , the relative signal variation between any two voxels in a window, can be quite varied. Previous works [28] address point irregularity only by using positional encoding in self-attention, but they are unaware of the variations of other signals. In Fig. 1, we illustrate sparse voxels in a window and signal irregularity on a 2D example.

3.2. SWIN3D Architecture

We have designed our SWIN3D backbone to address the issues mentioned above. It has a hierarchical structure similar to the Swin Transformer and is composed of the following modules: *voxelization*, *initial feature embedding*, SWIN3D *block*, and *downsample*. The *voxelization* module discretizes an input point cloud into a multiscale sparse voxel grid, the *initial feature embedding* module generates sparse voxel features at the finest voxel level for feature attention, the SWIN3D *block* performs memory-efficient self-attention on sparse voxel features within regular and shifted windows and addresses signal irregularity by *contextual relative signal encoding*, and the *downsample* module aggregates the sparse voxel features at l-th level to $(l+1)$-th level. SWIN3D contains 5-stage SWIN3D blocks, each of which operates at different voxel resolution. It serves as a multiscale feature encoder for any input point cloud and can be easily combined with task-specific decoders for various 3D tasks. Fig. 2 illustrates the architecture of our backbone. The details of each module are presented in the following section.

4. Module design of SWIN3D Transformer

4.1. Voxelization

Point cloud input An input point cloud is typically associated with point-wise signals, such as point position, point color and point normal. For a point $\boldsymbol{p}$, the concatenation of these signals is represented by $\boldsymbol{s_p} \in \mathbb{R}^m$. For any color component or normal component, it is mapped to the range $[-1, 1]$. A common setting is $m = 6$, meaning 3D point coordinates and RGB color.

Voxelization We employ sparse voxels as point proxies in our backbone. A 5-level hierarchical sparse voxel grid is constructed from the input point cloud, as shown in Fig. 2 with a 2D example. By default, the voxel size at the finest level is set to $2\,\mathrm{cm}$ for indoor scenes. The voxel size is doubled when the voxel level is increased by one. Point information is stored in the voxels from the finest level to the coarsest level in the following manner.

- For the voxel $\boldsymbol{v}$ at the finest level, we randomly select one point from within $\boldsymbol{v}$ and designate it as the *representative point* of $\boldsymbol{v}$, which is denoted by $\boldsymbol{r}_v$.
- For the voxel $\boldsymbol{v}$ at the $(l+1)$-th level, we first select all representative points from its child voxels. We then choose the representative point closest to the center of these points and make it the representative point of $\boldsymbol{v}$.

The voxelization step assigns unstructured points to structured sparse voxel grids, and the representative point is used to supply raw features to the initial feature embedding and provide contextual information for computing SWIN3D self-attention (see Section 4.3). For simplicity, we denote the signal at the representative point of voxel $\boldsymbol{v}$ as $\boldsymbol{s_v}$.

4.2. Initial feature embedding

Motivated by the observation that using a linear layer or MLP to project raw features to a high dimension does not yield good performance for Swin-like transformer architectures [28], we propose to lift the raw feature via sparse convolution. At the finest voxel level, we apply one layer of sparse convolution with a $3 \times 3 \times 3$ kernel, followed by a batch normalization (BN) and a ReLU layer, to transform

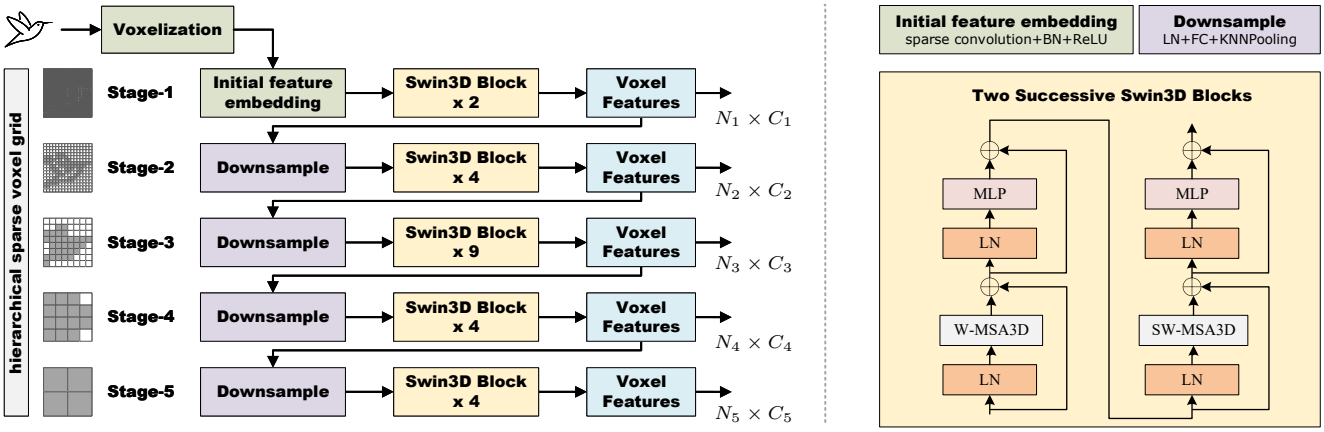

Figure 2. **Left**: The architecture of SWIN3D. It consists of five-stage transformer blocks that apply self-attention to sparse voxels within regular and shifted windows at various levels of a hierarchical sparse grid. The grids on the left are the illustration of sparse grids in 2D, with gray cells representing non-empty voxels. N_i denotes the number of sparse voxels at the i-th level, and C_i is the feature channel dimension. **Right**: The detailed operations of each module.

the input voxel feature to $\mathbb{R}^{C_1}$. The input feature on voxel $\boldsymbol{v}$ is set as the concatenation of the positional offset: $\boldsymbol{r}_v - \boldsymbol{c}_v$ and other point signals stored at $\boldsymbol{r}_v$, where $\boldsymbol{c}_v$ is the center of $\boldsymbol{v}$. We do not use the absolute point position because our goal is to learn local priors via convolution. Compared to KPConv [42] utilized by [28], our initial feature embedding is much lighter and five times faster.

4.3. SWIN3D block

Our SWIN3D block is based on the Swin Transformer block design: it operates on both regular and shifted windows in 3D. The voxel grid is split into non-overlapping windows at level l, with a window size of $M \times M \times M$. The shifted window is created by shifting the regular window with an offset of $(\lfloor \frac{M}{2} \rfloor, \lfloor \frac{M}{2} \rfloor, \lfloor \frac{M}{2} \rfloor)$. We modified the standard multi-head self-attention to improve memory efficiency and address signal irregularity, as described below. The number of heads is denoted by N_H.

4.3.1 Memory-efficient self-attention scheme

Vanilla self-attention For a given input window with N non-empty voxels, denoted by $\{\boldsymbol{v}_i\}_{i=1}^N$, the network features of these voxels are represented by $\{\boldsymbol{f}_i\}_{i=1}^N$. A vanilla multi-head self-attention is then applied to the input voxel features, which is a weighted sum of the projected voxel features:

$$\boldsymbol{f}^{\star}_{i,h} = \sum_{j=1}^{N} \alpha_{ij,h} \cdot \boldsymbol{f}_j \boldsymbol{W}_{V,h}, \qquad i = 1, \ldots, N, \qquad (1)$$

where $\{\boldsymbol{f}^{\star}_{i,h}\}_{i=1}^N$ are the output feature vectors at the h-th head. The weight coefficient $\alpha_{ij,h}$ is the SoftMax version of $e_{ij,h}$, *i.e.*, $\exp(e_{ij,h}) / \sum_{k=1}^{N} \exp(e_{ik,h})$, and $e_{ij,h}$ is in a scaled dot-product attention form:

$$e_{ij,h} = \frac{(\boldsymbol{f}_i \boldsymbol{W}_{Q,h})(\boldsymbol{f}_j \boldsymbol{W}_{K,h})^T}{\sqrt{d}}. \qquad (2)$$

Here, $\boldsymbol{W}_{Q,h}, \boldsymbol{W}_{K,h}, \boldsymbol{W}_{V,h}$ are linear projection matrices for *Query*, *Key* and *Value* computation, and d is the channel number of the h-th head.

Memory-efficient self-attention The common multi-head self-attention implementation requires two passes to calculate $\{\alpha_{ij,h}\}$. The first pass computes all $\exp(e_{ij})$s and the second pass accumulates their sums, *i.e.*, $\sum_{k=1}^{N} \exp(e_{ik})$. This necessitates storing all α_{ij}s for Eq. (1), resulting in $\mathcal{O}(N^2 \times N_H)$ memory complexity. For Swin-like Transformer architectures, this complexity is $\mathcal{O}(N^2 \times N_H \times N_w)$, where N_w is the number of windows. In the case of a 3D Swin Transformer with window size $M \times M \times M$, $N = \mathcal{O}(M^2)$ and N_w can be very large if the scale of the input point cloud varies greatly. This high memory cost in 3D limits the use of large windows and deeper networks; however, this is not a significant issue in the 2D Swin Transformer because N_w is at least one order smaller than its 3D counterpart, and the input image usually has a fixed size.

We observe that Eq. (1) can be rewritten in the following form:

$$\boldsymbol{f}^{\star}_{i,h} = \frac{\sum_{j=1}^{N} \left(\exp(e_{ij,h}) \boldsymbol{f}_j \boldsymbol{W}_{V,h}\right)}{\sum_{j=1}^{N} \exp(e_{ij,h})}, \qquad i = 1, \ldots, N, \qquad (3)$$

which allows us to postpone the SoftMax normalization and avoid constructing and storing $\{\alpha_{ij,h}\}$ explicitly. Therefore, we modify the second pass by calculating the denominator and numerator of Eq. (3) simultaneously. During computation, $\{\exp(e_{ij,h})\}_{j=1}^N$ for $\boldsymbol{f}^{\star}_{i,h}$ are calculated on the fly,

without storage. This eliminates the quadratic complexity of the memory cost of $\{\alpha_{ij,h}\}$. For gradient propagation, each $\exp(e_{ij,h})$ is computed twice during the training stage. However, this additional computation cost is negligible as the self-attention computation is a memory-intensive operation not a computation-intensive operation, thus our memory reduction does not slow down the computation and could reduce the execution latency as well (see evaluation in Section 6).

Contextual relative signal encoding

Swin Transformer utilizes *relative position bias* [41] to enhance the performance of its backbone. Wu *et al*. [52] proposed a novel contextual mode for relative positional encoding, referred to as *contextual relative position encoding* (cRPE), which adds relative position encoding to both queries and keys. Lai *et al*. [28] used cRPE to capture fine-grained position information in 3D self-attention computation. In our work, we extended cRPE to all kinds of signals, not just point positions, as other signals such as RGB color also display high variation within a window and these variations should be captured by self-attention. We refer to this generalized version as *contextual relative signal encoding*, or cRSE. The multi-head self-attention with cRSE is formulated as follows.

We first modify e_{ij} to include the difference between the voxel signals $\Delta \mathbf{s}_{ij} := \boldsymbol{s}_{\boldsymbol{v}_i} - \boldsymbol{s}_{\boldsymbol{v}_j}$:

$$e_{ij,h} = \frac{(\boldsymbol{f}_i \boldsymbol{W}_{Q,h})(\boldsymbol{f}_j \boldsymbol{W}_{K,h})^T + b_{ij,h}}{\sqrt{d}}. \quad (4)$$

Here, $b_{ij,h}$ is the contextual signal encoding:

$$b_{ij,h} = (\boldsymbol{f}_i \boldsymbol{W}_{Q,h})\big(\mathbf{t}_{K,h}(\Delta \mathbf{s}_{ij})\big)^T + (\boldsymbol{f}_j \boldsymbol{W}_{K,h})\big(\mathbf{t}_{Q,h}(\Delta \mathbf{s}_{ij})\big)^T. \quad (5)$$

The output of self-attention is revised to:

$$\boldsymbol{f}^{\star}_{i,h} = \frac{\sum_{j=1}^{N} \exp(e_{ij,h})(\boldsymbol{f}_j \boldsymbol{W}_{V,h} + \mathbf{t}_{V,h}(\Delta \mathbf{s}_{ij}))}{\sum_{j=1}^{N} \exp(e_{ij,h})}, \quad (6)$$

where $\mathbf{t}_{K,h}$, $\mathbf{t}_{Q,h}$ and $\mathbf{t}_{V,h}$ are trainable functions that map the signal differences to $\mathbb{R}^d$.

In order to make these trainable functions lightweight, we follow [41, 28] to quantify signal differences by a set of learnable look-up tables: $\{t^{Q,h}_1, \ldots, t^{Q,h}_m\}$, $\{t^{K,h}_1, \ldots, t^{K,h}_m\}$, and $\{t^{V,h}_1, \ldots, t^{V,h}_m\}$, each with a fixed length L_i, where $\star$ can be Q, K or V. For an input vector Δ, its table indices are determined by:

$$I_l(\Delta) = \lfloor \frac{(\Delta[l] - \texttt{minquat}[l])L_l}{\texttt{quat}[l]} \rfloor. \quad (7)$$

$\Delta[l]$ is the l-th component of Δ, $\texttt{quat}[l]$ and $\texttt{minquat}[l]$ are the quantification range and the lower bound of signal difference for the l-th signal, respectively. For common signal types, $\texttt{quat}[l]$ and $\texttt{minquat}[l]$ are defined as follows:

- If the l-th signal corresponds to point position, $\texttt{quat}[l] = 2h$ and $\texttt{minquat}[l] = -h$, where h is the physical height of the cubic window.
- If the l-th signal corresponds to one of the RGB components, $\texttt{quat}[l] = 2$ and $\texttt{minquat}[l] = -1$.
- If the l-th signal corresponds to one of the point normal components, $\texttt{quat}[l] = 2$ and $\texttt{minquat}[l] = -1$.

With the look-up tables and index functions, we have $\mathbf{t}_{Q,h}(\Delta) = \sum_{l=1}^{m} t^{Q}_{l,h}[I_l(\Delta)]$, $\mathbf{t}_{K,h}(\Delta) = \sum_{l=1}^{m} t^{Q}_{l,h}[I_l(\Delta)]$, $\mathbf{t}_{V,h}(\Delta) = \sum_{l=1}^{m} t^{V}_{l,h}[I_l(\Delta)]$. The use of look-up tables for cRSE introduces additional $3\sum_{i=1}^{m} L_i \times N_H$ parameters. By default, we set $L_l = 4$ if the l-th signal corresponds to point position, and $L_l = 16$ if the l-th signal corresponds to color or normal components. The effectiveness of cRSE is evaluated extensively in Section 6.4.

Transformer block

Our SWIN3D transformer block, denoted by `S-MSA3D` (on regular windows) and `SW-MSA3D` (on shifted windows), is composed of the revised multi-head self-attention along with other transformer components such as LayerNorm and MLP layer, as illustrated in Fig. 2-right.

Efficient implementation

We revised the self-attention module of Stratified Transformer [28] to improve its efficiency. The revision includes optimizing kernel scheduling, enabling half-precision, and reducing accesses of atomic operations, and we call this revision our vanilla implementation. We further extend cRPE to cRSE and integrate our memory-efficient design. More details on the implementation are presented in Appendix.

Downsample

The voxel features at the l-th level are downsampled to the $(l+1)$-th level by first lifting all sparse voxel features to $\mathbb{R}^{C_{l+1}}$ through a LayerNorm and an FC layer. Then, for any sparse voxel $\boldsymbol{v}$ at the $(l+1)$-th level, the features of its k-nearest voxels at the l-th level are maxpooled and assigned to $\boldsymbol{v}$. This downsampling strategy is referred to as `KNNPooling`, with k set to 16 by default.

5. SWIN3D Backbone pretraining

Backbone models

We created two versions of SWIN3D: SWIN3D-S and SWIN3D-L. The window size for the first stage is set to $5 \times$

Stratified Transformer

(Improved version of Stratified Transformer (where we do kernel scheduling, half-precision and reduce atomic operation. and cRSE) . But it does not include (Memory-efficient-self-attention that we have in our SWIN3D.

		Impl. of [28]		Our-Vanilla		Our-Efficient	
Block	**#Pts**	**Time**	**Mem.**	**Time**	**Mem.**	**Time**	**Mem.**
Stage-1	109.48 k	487.7	1380.1	25.7	555.4	**20.3**	**268.68**
Stage-2	15.05 k	122.9	467.4	16.0	180.4	**14.1**	**95.4**
Stage-3	4.01 k	56.5	233.6	9.3	104.8	**7.1**	**47.9**
Stage-4	1.01 k	29.0	120.5	6.5	60.0	**4.8**	**24.4**
Stage-5	0.25 k	9.6	36.7	4.3	21.2	**2.4**	**8.7**

Table 1. We compare the efficiency of self-attention by reporting the average statistics of point number (same to the number of non-empty voxels), execution time (in milliseconds) and memory footprint (in megabytes) for a single forward-backward iteration. The implementation of [28] only works with double-precision.

Reduce spatial resolution in down sampling, For this we need(3,2,2,2) strides

5×5 and for the rest stages, it is $7 \times 7 \times 7$. The layer numbers are $\{2, 4, 9, 4, 4\}$ with downsample strides $\{3, 2, 2, 2\}$. The feature dimensions (#FD) and head numbers (#HD) at each stage are:

- **SWIN3D-S**: #FD = $\{48, 96, 192, 384, 384\}$, #HD = $\{6, 6, 12, 24, 24\}$;
- **SWIN3D-L**: #FD = $\{80, 160, 320, 640, 640\}$, #HD = $\{10, 10, 20, 40, 40\}$.

positional{x,y,z}
color{255,255,255}

By default, the input point signal contains positional and color information only. When the input signal contains point normals and cRSE uses normal signals, we use SWIN3D$_n$-S and SWIN3D$_n$-L to denote our backbone models. The efficiency and support for large models of our backbone design is determined by the following assessments.

Model efficiency We evaluated the memory and computation efficiency of our backbone model by computing the average memory usage and computational time based on 70 point clouds from ScanNet [11] (see Table 1). Our SWIN3D-S model is referred to as *Our-Efficient*. We also compared it with (1) Stratified Transformer [28], which also adapted the Swin Transformer for 3D point clouds; (2) an improved version of Stratified Transformer based on our revision described in Section 4.3.4 and contextual relative signal encoding, excluding our memory-efficient self-attention scheme, referred to as *Our-Vanilla* implementation. All models had the same number of Transformer blocks and the same amount of Transformer parameters. Compared with [28], our vanilla implementation already significantly decreased computational and memory cost and our memory-efficient self-attention further reduced half of memory consumption and sped up the computation. The memory and computation efficiency of our design allowed us to explore large SWIN3D models to take advantage of large data sets.

Scalability:

Support to large models We assessed the scalability of our design by examining its GPU memory consumption when applied to large models. We used SWIN3D-S as the base model and tested its GPU memory utilization on ScanNet data, increasing the width and depth of the network, the number of heads, and the window size. Additionally, we compared the results to the vanilla version of our model, which does not employ memory-efficient self-attention. The experiment setup is as follows:

Increase #FD

- **Wider networks**: We created five models based on SWIN3D-S by increasing the feature channels with five different ratios: $\frac{2}{3}, 1, \frac{4}{3}, \frac{5}{3}, 2$, respectively. The model with ratio $\frac{5}{3}$ is equivalent to SWIN3D-L.
- **Deeper networks**: We created five models based on SWIN3D-S by changing the number of Swin block layers at Stage-3 to 6, 9, 12, 15, 18, respectively. Here, the default SWIN3D-S uses 9 layers.
- **Large head number** We created five models based on SWIN3D-S by increasing the number of heads by four different factors: 1, 2, 4, 8.
- **Large window size** We tested different window sizes: $5^3, 7^3, 9^3, 11^3, 13^3, 15^3$.

We present the GPU memory consumption of the test in Fig. 3. We can observe that as the channel number and the block number are proportional to memory storage, the rate of memory usage is almost constant; and the models with our memory-efficient self-attention save around 25% memory compared to the models with the vanilla-version self-attention. The quadratic memory saving by our memory-efficient self-attention is clearly visible when increasing the head number or the window size, compared to the models with the vanilla-version self-attention. In conclusion, the reduction of GPU memory enabled by our memory-efficient self-attention makes our backbone architecture suitable for designing large backbones.

From the test, it is evident that our backbone design can accommodate models with greater capacity than SWIN3D-L. However, we found experimentally that more large models tend to overfit our pre-training data set, so we did not explore them further in this study.

5. Backbone pretraining

Pretraining data preparation We opt to pretrain our backbones with the Structured3D dataset [65], which contains 21835 rooms in 3500 **synthetic** scenes and is equipped with a variety of high-quality 3D objects and layouts. This dataset is one-order larger than other **real** indoor datasets such as MatterportLayout [66] (2295 rooms) and ScanNet [11] (1613 rooms). We use the provided RGBD and panoramic images to generate the training data as follows. RGB -> color space and D-> Depth

We acquire all RGBD and panoramic images associated with the room using the official room description, and project RGB and semantic labels of all images into 3D points

panoramic images: The image capture 360 degree sence of an image**
In our modern mobile camera we have this panoramic settings.

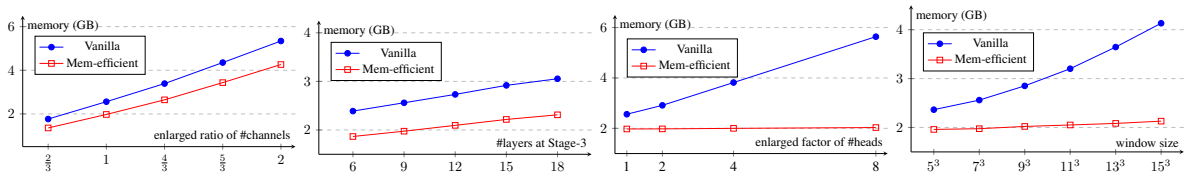

Figure 3. Evaluation on the support to large models. **Top-left**: GPU memory consumption of wider networks. **Top-right**: GPU memory consumption of deeper networks. **Bottom-left**: GPU memory consumption with respect to head numbers. **Bottom-right**: GPU memory consumption with respect to window size. The GPU memory was measured on a forward-and-backward iteration, averaged on all ScanNet data.

semantic level

Datasets	Task	#C	wall	floor	cabinet	bed	chair	sofa	table	door	window	bookshelf	bookcase	picture	counter	desk	shelves	curtain	dresser	pillow	mirror	ceiling	refrigerator	television	shower curtain	nightstand	toilet	sink	lamp	bathtub	garbagebin	board	beam	column	clutter	otherstructure	otherfurniture	otherprop.
Structured3D	Seg.	25	✓	✓	✓	✓	✓	✓	✓	✓	✓			✓		✓	✓	✓	✓	✓	✓	✓	✓	✓		✓		✓	✓							✓	✓	✓
ScanNet	Seg.	20	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓		✓	✓	✓		✓					✓		✓		✓	✓		✓							✓	
	Det.	18			✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓		✓					✓		✓		✓	✓		✓	✓							
S3DIS	Seg.	13	✓	✓			✓	✓	✓	✓	✓		✓									✓									✓	✓	✓	✓			
	Det.	5					✓	✓	✓				✓																		✓						

Table 2. We provide a list of semantic labels for our Structured3D pretraining dataset, as well as the datasets of the downstream tasks. The list is denoted by "#C", which stands for the number of segmentation labels. Additionally, "Seg." and "Det." indicate the tasks of semantic segmentation and 3D detection, respectively.

Intrinsic--> (camer's focal-length,lens type etc.), extrinsic -> camera position in 3D space etc.

based on their intrinsic and extrinsic camera parameters. To reduce the large number of points, we divide the space into cubes of $1\,\mathrm{cm}^3$ and keep only one point in each cube. The remaining points form the point cloud of the room. We also estimate point normals from depth maps for training SWIN3D$_n$ models.

"Point Normal Means" For each point, we have a vector perpendicular(normal) to that point . This normlas is mapped from depth maps. And input of Swin3Dn

Backbone pretraining We selected 3D semantic segmentation as our pre-training task, with 25 semantic labels. Four original segmentation labels (counter, box, toilet, bathtub) were excluded due to their rarity. Table 2 shows the segmentation labels used in pretraining, as well as those in the downstream tasks of ScanNet segmentation and detection, and S3DIS segmentation and detection. There is some overlap between our pre-training data and the datasets of the downstream tasks, and some labels used in the downstream tasks are not present in our pre-training data.

semantic segmenation is an computer vision problem. 25 semantic lables. means 25 image for segmentation

We follow the original data split: 18349 rooms for training, 1776 rooms for validation, and 1691 rooms for testing. We use SWIN3D as the encoder and a simple decoder to output semantic labels of input points. The decoder is similar to the UNet decoder. We upsample the features from the coarsest level using interpolation, followed by a Linear Layer to match the dimensions. We then add fine-level features from the encoder using skip-connection. The purpose of the simple decoder is to make the backbone the main factor in feature learning. The network was trained with 100 epochs, a batch size of 12, and augmented input data through random cropping and rotation. We use the AdamW optimizer with a Cosine learning rate scheduler. Table 3 reports the network parameters, the amortized inference latency measured in the Structured3D validation set, and the average and peak memory footprint of network training in a subset (600 samples) of the training dataset. Training SWIN3D-S and SWIN3D-L took 488 and 703 GPU hours with NVidia V100 GPUs, respectively. We also pretrained SWIN3D$_n$-S and SWIN3D$_n$-L and use them only for the ScanNet segmentation task. During the network training phase, we employed the data cropping strategy of [28] to randomly crop a portion from the input scene for training, with a maximum of 120 000 sparse voxels. Additionally, we use the data augmentation technique of [28] for network training.

zooming with interpolation

skip connection use in U-NET

Model	Params	Latency	Memory (Avg./Peak)
SWIN3D-S	23.57 M	377.98 ms	2.24/3.69 GB
SWIN3D-L	60.75 M	554.58 ms	4.11/6.73 GB

Table 3. Model parameters, inference latency, and memory footprint, evaluated on Structured3D segmentation.

Fine-tuning for downstream tasks We employ our pretrained backbone as a multi-resolution feature encoder and attach it to a task-specific decoder for downstream tasks. The pretrained network weights and look-up tables are loaded for initialization, while the decoder weight is randomly initialized. Our experiments on downstream tasks are presented in Section 6.

encoder: Extract the input feature
decoder: Use extracted feature for making final output like, `Sementatic level`.

Hyperparameter

Cosine learning rate: reduce the value of learning rate. like: cos 0=1, cos30 = root(3)/2, cos 45 = 1/root(2) like this

6.1.2: For downstream tasks: we replace many object detection models (ScanNet, S3DIS, FCAF3D and CAGroup3D), encoder (where we extract feature) with Swin3D (also extract feature) From decoder we get the final output. ***(basically, in deep learning in many computer vision model have encoder and decoder . Swin3D can act as an encoder. So, Replace the other model encoder with our cute Swin3D.

6. Experimental Analysis

We conducted experiments to assess the scalability, generality, and effectiveness of our backbone design for typical indoor scene understanding tasks. This section is structured as follows: we first present the experimental setup of downstream tasks in Section 6.1, then evaluate our model's capability through a series of experiments, comparisons, and ablation studies in Sections 6.2 to 6.4.

6.1. Experimental settings for downstream tasks

6.1.1 Semantic segmentation

Datasets ScanNet [11] contains 1613 indoor scans with 20 common semantic labels. We followed the original training/validation/test data split and reported the mean Intersection over Union (IoU) on the validation and test datasets. S3DIS contains 272 rooms from 6 large-scale areas. One area was selected as the validation set, and the other areas were used for training. We reported mean IoU on Area-5 and 6-fold cross-validation results.

Matrix for segmentation task.

6-fold cross-validation, we divide the dataset into 6 parts. 5 for testing and 1 for testing.

Networks To adapt our pretrained encoders to the segmentation task, we revised the decoder structure used for our pretraining by adding a SWIN3D block at each level after the skip-connection to improve the decoder capability. The minimum voxel size for ScanNet and S3DIS is 2 cm and 5 cm, respectively. As ScanNet point clouds possess point normal information and many existing methods utilized it, we used SWIN3D$_n$-S and SWIN3D$_n$-L for this task.

Training During the training stage, we augmented the data through random cropping, scaling, and rotation. We fine-tuned our models (SWIN3D-S and SWIN3D-L) for 600 epochs for both ScanNet and S3DIS, with a batch size of 12.

6.1.2 3D detection

Datasets The ScanNet [11] dataset contains labels for 18 objects, with 1201 scans used for training and 312 for validation. The S3DIS [1] dataset contains instance labels of 7 categories, including floor and ceiling. Following FCAF3D [40], we excluded these two categories and trained and validated our models on the remaining five categories. AP@0.25 and AP@0.5 are the evaluation metrics employed in our experiment.

AP@0.25 and AP@0.5, AP(Average Precision) evolution matrix for object detection.

Networks We replaced the encoders of two state-of-the-art 3D detection networks (FCAF3D [40] and CAGroup3D [45]) with our pretrained SWIN3D to demonstrate the advantages of using our pretrained backbones for 3D detection. Specifically, For FCAF3D, we replaced the Sparse-Convolution-based ResNet in FCAF3D with our pretrained SWIN3D and kept the other modules unchanged, which we named SWIN3D-S+FCAF3D and SWIN3D-L+FCAF3D; for CAGroup3D, we replaced its feature extractor (BiResNet) with our pretrained encoder with upsample layers, and kept the other modules for proposal generation and the detection head unchanged, which we named SWIN3D-S+CAGroup3D and SWIN3D-L+CAGroup3D.

Training For network training, we set the finest voxel size to 2 cm for both ScanNet and S3DIS, and use the same data augmentation as CAGroup3D and FCAF3D. We adjusted our SWIN3D-S+CAGroup3D and SWIN3D-S+FCAF3D models for ScanNet 3D detection by training them for 200 epochs with a batch size of 12. Similarly, for S3DIS 3D detection, we fine-tuned our SWIN3D-L+FCAF3D model for 200 epochs with a batch size of 8.

6.2.1: test-time augmentation in evalution:

6.2. Performance on downstream tasks

6.2.1 Semantic segmentation

We quantitatively evaluated our pretrained models with fine-tuning on the semantic segmentation task and compared them to state-of-the-art methods, as shown in Table 4. Following existing approaches such as Stratified-Transformer [28], PointTransformerV2 [55], O-CNN [47], OctFormer [46] that perform test-time augmentation in evaluating segmentation performance, we voted our segmentation results via 12 rotation augmentation. For reference, we also report the unvoted results of several methods, including ours. The unvoted results are reported in parentheses. In the following, we analyze the results in detail.

unvoted results: result we got without doing test-time augmentation.

Comparison with supervised methods Among the existing supervised methods, PointTransformerV2 [55], O-CNN [47], OctFormer [46] utilize point normals on ScanNet segmentation. Our SWIN3D-S outperforms the best supervised method by 0.4 mIoU on ScanNet val, +0.8 mIoU on S3DIS Area5, and +0.6 mIoU on S3DIS 6-fold. With greater capacity, our SWIN3D-L surpasses all the compared supervised methods with 1.0 mIoU on ScanNet val, +2.3 mIoU on S3DIS Area5, and +2.2 mIoU on S3DIS 6-fold. With point normal inputs for pre-training and fine-tuning, our SWIN3D$_n$-L further enhances performance on ScanNet val by +0.8 mIoU. We also present the results of SWIN3D$_n$-S on ScanNet, which is +0.7 mIoU better than SWIN3D-S.

Metrix for segmentation task: mIoU(mean Intersection over Union)

Comparison with pretraining methods We compared our approach to existing unsupervised pretraining methods, such as PointContrast [57], SceneContext [23], DepthContrast [63] and MaskContrast [56], which use multi-view data from ScanNet for pretraining and Minkowski U-Net [9] as the encoder architecture. MaskConstrast also combines the ArkitScenes dataset [3] with ScanNet for pretraining. As can be seen in Table 4 (lower section), these unsupervised

(semantic Segmentation task Metrix Result:)

Method	ScanNet Segmentation		S3DIS Segmentation	
	Val mIoU (%)	Test mIoU (%)	Area5 mIoU (%)	6-fold mIoU (%)
FastPointTransformer [36]	72.4	-	70.4	-
PointMetaBase-XXL [30]	72.8	71.4	72.0	77.0
LargeKernel3D [7]	73.5	73.9	-	-
Mix3D [34]	73.6	**78.1**	-	-
Stratified Transformer [28]	74.3(73.1)	74.7	72.0	-
PointConvFormer [54]	74.5	74.9	-	-
O-CNN [47]	74.5	76.2	-	-
PointTransformerV2 [55]	75.4(74.4)	75.2	71.6	-
OctFormer [46]	75.7(74.5)	76.6		-
PointNeXt-XL [37]	-	-	71.1	74.9
PointMixer [8]	-	-	71.4	-
WindowNorm [49]	-	-	72.2	**77.6**
SWIN3D-S*	75.5(74.5)	-	**72.5**	76.9
SWIN3D$_n$-S*	**76.4**(75.2)	-	-	-
DepthContrast [63]	71.2	-	70.6	-
SceneContext [23]	73.8	-	72.2	-
PointContrast [57]	74.1	-	70.9	-
MaskContrast [56]	75.5(74.4)	-	-	-
SWIN3D-S	76.1(75.0)	-	73.0	78.2
SWIN3D$_n$-S	76.8(75.7)	-	-	-
SWIN3D-L	76.7(75.7)	-	**74.5**	**79.8**
SWIN3D$_n$-L	**77.5**(76.4)	77.9	-	-

Unsupervised

Table 4. Quantitative evaluation on semantic segmentation. The methods in the upper part of the table are supervised methods, while those in the lower part are based on pre-training. We present the highest scores achieved by the compared approaches. On the ScanNet benchmark (test dataset), we ensembled the results of three trained models by voting the prediction on over-segmented meshes, similar to Mix3D. The paper of WindowNorm is a concurrent and unpublished work.

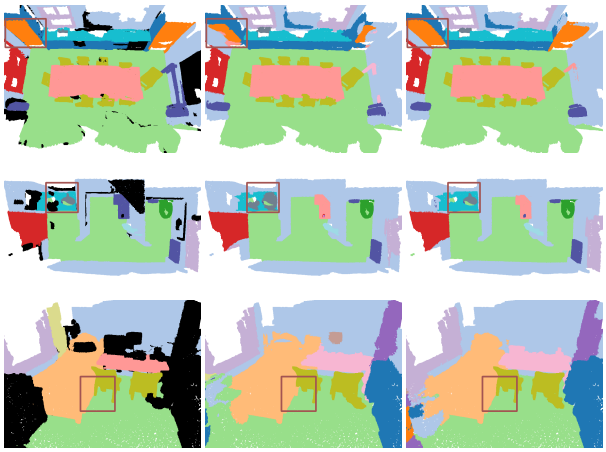

Figure 4. Visual comparison of ScanNet segmentation. **Left**: Ground truth segmentation labels (points color in black are not labeled in the original dataset). **Middle**: Stratified Transformer's results. **Right**: SWIN3D$_n$-L's results.

pretrained methods have lower performance than state-of-the-art supervised methods, with large gaps to our approach. One may wonder if unsupervised pretrained methods can take advantage of large datasets. Our initial tests (see Section 6.3) indicate that the advantages are restricted, probably because the convolutional encoder structure is not able to effectively extract data priors in comparison to the transformer structure.

In unsupervised problem we don't get good result, may be encoder data priors.

Here, we do model training from scratch to solve downstream work.

Non-pretrained SWIN3D We also evaluated our backbone structure without pretraining, *i.e.*, training SWIN3D from scratch for downstream tasks. e also trained SWIN3D-S from scratch for comparison (600 epochs for ScanNet and 3000 epochs for S3DIS), denoted by SWIN3D-S* and SWIN3D$_n$-S*. SWIN3D-S* has a comparative performance to existing works, only inferior to WindowNorm [49] on S3DIS 6-fold. SWIN3D$_n$-S* can further improve SWIN3D-S* due to the use of point normals. Despite their good performance, they are consistently inferior to those of their pre-trained versions, demonstrating the effectiveness of pre-training.

We noticed that SWIN3D-L trained from scratch does not produce excellent results. SWIN3D-L* and SWIN3D$_n$-L* only achieved 73.9 mIoU and 74.2 mIoU on ScanNet segmentation (val), respectively. This is due to the overfitting caused by the large model size in comparison to the size of the training data (ScanNet or S3DIS). This is in agreement with previous findings in the NLP and vision fields (*e.g.*, Bert, SwinTransformerV2) that large transformer models need a considerable amount of data for pretraining to show their advantages.

Additional results In Table 5 and Table 6, we further provide semantic segmentation performance reports on all categories of ScanNet and S3DIS. For 6-fold cross validation of S3DIS, we fine-tuned our models for 600 epochs only.

Method	mIoU	wall	floor	cabinet	bed	chair	sofa	table	door	window	bookshelf	picture	counter	desk	curtain	refri.	shower cur.	toilet	sink	bathtub	other.
Stratified Transformer [28]	74.3	86.2	95.1	66.4	80.9	90.0	82.9	76.6	69.0	71.7	82.2	30.1	66.0	71.1	75.8	66.5	68.6	94.6	67.6	88.3	56.3
PointTransformerV2 [55]	75.4	86.1	**95.4**	67.4	**81.9**	91.9	**86.5**	77.5	68.8	68.7	84.5	34.1	66.7	71.1	78.7	69.2	71.2	94.5	65.6	**89.1**	60.5
SWIN3D$_n$-S	76.8	**87.6**	95.1	67.0	79.3	91.6	83.7	78.2	69.6	71.0	86.8	**39.9**	68.1	**73.6**	80.2	72.6	75.8	**95.6**	**71.2**	**89.7**	60.2
SWIN3D$_n$-L	**77.5**	87.0	95.0	**73.2**	81.5	**92.1**	84.2	**79.1**	**70.6**	**72.8**	**87.3**	36.9	**68.3**	73.3	**80.9**	**73.4**	**77.8**	95.4	70.9	88.6	**60.8**

Table 5. Category-wise segmentation results evaluated on ScanNet validation set.

mAP -> mean Average Precision .

Method	6-fold mIoU	A1	A2	A3	A4	A5	A6
SWIN3D-S	78.2	**82.2**	64.2	83.7	75.9	73.0	86.3
SWIN3D-L	**79.8**	82.1	66.3	**85.9**	**76.7**	**73.4**	**87.2**

Table 6. 6-fold S3DIS segmentation results. "An" means that the n-th Area is the test data and other 5 Areas are used for training. The reported number is mIoU.

Method	mAP@0.25	mAP@0.5
RepSurf [38]	71.2	54.8
FCAF3D [40]	71.5	57.3
SoftGroup [44]	71.6	59.4
CAGroup3D [45]	**75.1**	**61.3**
SWIN3D-S*+FCAF3D	72.1	56.8
SWIN3D-S*+CAGoup3D	73.3	58.6
Point-M2AE [62]	50.1	33.2
PointContrast [57]	59.2	37.3
RandomRooms [39]	68.6	51.5
SWIN3D-S+FCAF3D	74.2	59.5
SWIN3D-L+FCAF3D	74.2	58.6
SWIN3D-S+CAGoup3D	76.4	62.7
SWIN3D-L+CAGoup3D	**76.4**	**63.2**

Table 7. Quantitative evaluation on 3D detection (ScanNet). The methods in the upper part of the table are supervised methods, while those in the lower part are based on pretraining.

And in the test phase, we voted our predictions across 12 rotation augmentations. As for Area 5, we can achieve 74.5 mIoU (voted) by fine-tuning our model for 3000 epochs. For visualization, we illustrate three segmentation results in Fig. 4.

6.2.2 3D detection

We quantitatively evaluated our pretrained models with fine-tuning on the 3D detection task and compared them to state-of-the-art methods, as shown in Tables 7 and 8. In the following, we analyze the results in detail.

Comparison with supervised methods On ScanNet 3D detection (as shown in Table 7), SWIN3D-S+FCAF3D and SWIN3D-L+FCAF3D both outperform FCAF3D, with an increase of 2.7 points in mAP@0.25. SWIN3D-S+CAGroup3D and SWIN3D-L+CAGroup3D both surpass CAGroup3D, with SWIN3D-L+CAGroup3D achieving a new record on mAP@0.50. Table 8 shows the performance

Method	mAP@0.25	mAP@0.5
GSDN [18]	47.8	25.1
FCAF3D [40]	**66.7**	**45.9**
SWIN3D-S*+FCAF3D	64.6	40.7
SWIN3D-S+FCAF3D	69.9	50.2
SWIN3D-L+FCAF3D	72.1	54.0
SWIN3D-S+FCAF3D(2-stage)	**75.4**	**58.6**

Table 8. Quantitative evaluation on 3D detection (S3DIS).

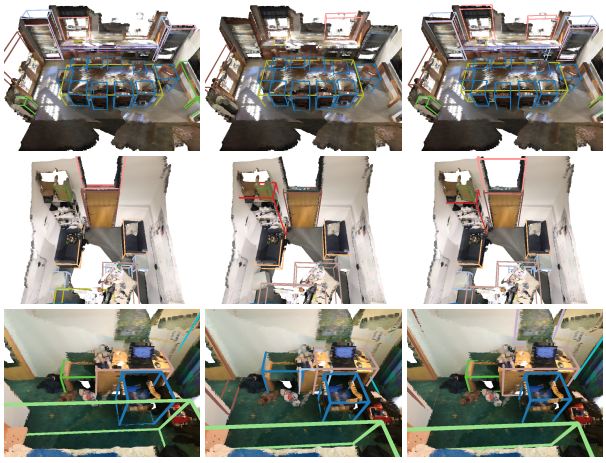

Figure 5. Visual comparison of ScanNet 3D detection. **Left**: Ground truth. **Middle**: FCAF3D's result. **Right**: SWIN3D-L+CAGroup3D's results. Note that the original CAGroup3D paper does not release its checkpoint, so no visual results provided in this figure.

of our model on S3DIS. Compared to FCAF3D, our pre-trained backbones significantly improve the performance, with SWIN3D-S+FCAF3D increasing by 4.3 points and SWIN3D-L+FCAF3D increasing by 8.1 points in mAP@0.5. The experiments of detection lead us to the conclusion that semantic segmentation is an effective pretext task for 3D pretraining, as our pretrained backbone demonstrates remarkable generality to the 3D detection task.

Two-stage finetuning We discovered through experimentation that S3DIS detection can be improved by a two-step finetuning process. Initially, we fine-tuned SWIN3D-S on ScanNet detection. Then, we loaded the fine-tuned SWIN3D

Method	Pre.	mAP@0.5	cabinet	bed	chair	sofa	table	door	window	bookshelf	picture	counter	desk	curtain	refri.	shower cur.	toilet	sink	bathtub	other.
FCAF3D[40]	✗	57.3	35.8	81.5	89.8	85.0	62.0	44.1	30.7	58.4	17.9	31.3	53.4	44.2	46.8	**64.2**	91.6	52.6	84.5	57.1
CAGroup3D[45]	✗	61.3	41.4	82.8	90.8	**85.6**	64.9	54.3	37.3	64.1	31.4	41.1	**63.6**	44.4	57.0	49.3	98.2	**55.4**	82.4	58.8
SWIN3D-S+CAGroup3D	✓	62.7	45.7	82.7	**91.0**	79.5	**67.4**	**57.5**	**42.7**	**59.8**	36.8	40.4	62.6	**48.2**	**60.7**	59.8	**99.7**	55.1	77.6	60.7
SWIN3D-L+CAGroup3D	✓	**63.2**	**46.1**	**85.5**	**91.0**	81.1	64.5	52.9	42.4	57.8	**38.2**	**47.2**	63.4	46.0	59.3	61.7	98.3	54.2	**85.4**	**63.6**

Table 9. Quantitative comparison on 3D detection (ScanNet).

Method	Pre.	mAP@0.5	table	chair	sofa	bookcase	board
GSDN [19]	✗	25.1	36.6	75.3	6.1	6.5	1.2
FCAF3D[40]	✗	45.9	45.4	88.3	70.1	19.5	5.6
SWIN3D-S	✓	50.2	52.8	**90.4**	78.8	**20.9**	7.9
SWIN3D-L	✓	**54.0**	**56.2**	90.3	**95.1**	19.6	**8.9**

Table 10. Quantitative comparison on 3D detection (S3DIS).

Figure 6. Visual comparison on S3DIS 3D detection. **Left**: Ground-truth 3D bounding boxes. **Middle**: FCAF3D's detection result. **Right**: SWIN3D-L+FCAF3D's detection results. Our SWIN3D-L+FCAF3D generates more compact and accurate proposals of table, sofa, and board, which are consistent with the result shown in Table 10.

and continued the fine-tuning on S3DIS training data, taking advantage of the prior knowledge gained from real data in the first step. The improved performance is reported in Table 8.

Comparison with pretrained methods We compared our approach to three existing unsupervised pretraining methods: PointContrast [57], Point-M2AE [62] and RandomRooms [39]. PointContrast employed ScanNet as its pretraining dataset and Minkowski U-Net as its encoder, Point-M2AE utilized both ShapeNet [5] and ScanNet as its pretraining dataset and a transformer encoder, and RandomRooms adopted random samples of multiple objects from ShapeNet as its pretraining dataset and PointNet++ as its backbone. In line with the observations in semantic segmentation, these unsupervised pretraining methods were unable to compete with supervised methods and our approaches.

Non-pretrained SWIN3D Without pretraining, our model SWIN3D-S* that were trained from scratch show average results, however, they are not as good as the state-of-the-art methods. We speculate that more training data for these tasks is required for training our transformer from scratch. The higher performance of our pretrained SWIN3D also reflect this fact.

Additional results In Tables 9 and 10, we present the category-wise performance report on ScanNet and S3DIS detection. Visualizations of some 3D detection results can be seen in Figs. 5 and 6.

6.3. Model scalability

Our backbone design takes advantage of large pretrained datasets, particularly for our large model SWIN3D-L. To evaluate the impact of the amount of pretrained data on the performance of downstream tasks with respect to backbone architectures, we conducted the following experiments.

Scalability of SWIN3D We pretrained our backbones with different amounts of training data: 10%, 33%, and 100%. We then fine-tuned them for downstream tasks. Fig. 7 shows the segmentation performance of our SWIN3D-S and SWIN3D-L models on the test dataset of Structured3D (left) and the validation set of ScanNet segmentation (right). The plots demonstrate that (1) with more training data, the performance of both models increases significantly; (2) SWIN3D-L has more capacity to benefit from large data and performs better than SWIN3D-S.

Scalability on the use of cRSE We pretrained our backbones with cRPE instead of our cRSE, referred to as SWIN3D-S$^+$ and SWIN3D-L$^+$. We then fine-tuned them on the ScanNet segmentation task, and the results are shown in the right side of Fig. 7. The plots show clearly that pretraining with our cRSE scheme exhibits better scalability than using cRPE. We also found that SWIN3D-S outperforms

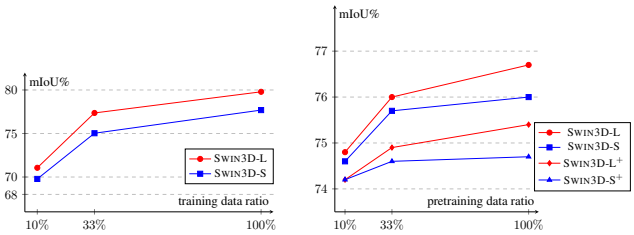

Figure 7. Model scalability with respect to different ratios of data for pretraining. **Left**: Test on Structured3D segmentation. **Right**: SWIN3D-L on downstream ScanNet segmentation. SWIN3D-L$^+$ and SWIN3D-S$^+$ stand for the models pretrained and fine-tuned with cRPE instead of cRSE.

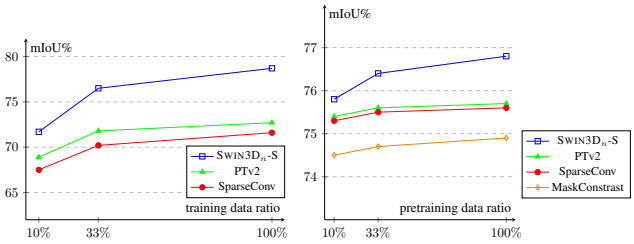

Figure 8. We tested the scalability of different 3D backbones with respect to different ratios of data for pretraining. "PTv2" refers to PointTranformerV2. **Left**: The results for Structured3D segmentation. **Right**: The results for ScanNet segmentation (val).

SWIN3D-L$^+$, which indicates that the capture of irregular signals is essential for backbone learning.

Scalability of other backbone architectures We selected a few representative backbone architectures, such as SparseConv Net used by MaskContrast [56] and PointTransformerV2, and pre-trained them on the Structured3D dataset with a segmentation pretext task similar to ours. Additionally, we retrained MaskContrast in its unsupervised manner with the Structured3D dataset. All these pretrained backbones were finetuned on the ScanNet segmentation task. Noted that for fair comparison, all these backbones used both color and normal signals for input in pre-training and fine-tuning stages. Their performance with respect to different amounts of pre-training data: 10%, 33%, and 100% are plotted in Fig. 8. We discovered that these backbones had limited advantages from large datasets and had a large performance gap compared to our SWIN3D-S, which has similar or fewer network parameters than theirs. The results of this experiment demonstrate that simply increasing the amount of data used for pre-training does not necessarily lead to significant performance improvement for some existing 3D backbones.

6.4. Ablation study

Dataset for pretraining We contrasted our backbones that had been pre-trained on either ScanNet or Structured3D for the downstream S3DIS segmentation task. Table 11 clearly demonstrates that our backbones gain more from the extensive synthetic Structured3D data than from the real but limited ScanNet data. This implies that the size of the training data is the most critical factor in 3D backbone training, despite the disparity between real and synthetic data.

Backbone	Pre. dataset	Area5 mIoU (%)	6-fold mIoU (%)
SWIN3D-S	ScanNet	71.8	76.7
SWIN3D-S	Structured3D	**73.0**	**78.2**
SWIN3D-L	ScanNet	68.9	73.6
SWIN3D-L	Structured3D	**74.5**	**79.8**

Table 11. S3DIS segmentation results by using the backbones pretrained on different datasets.

Input Point Signal	cRSE	Val mIoU (%)
pos+color	pos	73.1
pos+color	pos+color	**74.5**
pos+color+normal	pos	73.1
pos+color+normal	pos+color	74.4
pos+color+normal	pos+color+normal	**75.2**

Table 12. Efficacy evaluation of cRSE on ScanNet segmentation.

Backbone	loaded look-up tables	Val mIoU (%)
SWIN3D$_n$-L	pos	75.4
SWIN3D$_n$-L	pos+color	75.9
SWIN3D$_n$-L	pos+color+normal	**76.4**

Table 13. Ablation study of using the pretrained look-up tables on ScanNet segmentation.

Efficacy of cRSE We conducted two experiments to assess the effectiveness of cRSE. In the first, we trained SWIN3D-S from scratch in three different configurations: (1) using cRSE on point position only, similar to [28]; (2) using cRSE on point position and color; (3) using cRSE on point position, color, and point normal. We also either excluded or included the normal information from the input point cloud for training and testing. The results in Table 12 demonstrate that taking into account color and normal variation via cRSE can significantly enhance the performance. In the second experiment, we used the pretrained SWIN3D$_n$-L for ScanNet segmentation but did not load the pretrained look-up tables for color and/or normal components and trained these unloaded look-up tables from scratch during the network fine-tuning stage. The results in Table 13 show that the use of pretrained tables is essential for improved performance.

7. Conclusion

In this paper, we introduce a novel 3D backbone — SWIN3D for indoor scene understanding, which demonstrates its scalability, generality, and superior performance through extensive experiments. We identify several possible directions for future research that can further exploit the potential of SWIN3D. First, we would like to explore

self-supervised pretraining methods using our backbone and enrich its representation with more real and synthetic data, including outdoor 3D data. Second, we propose to leverage image data and to integrate both pretrained image backbones and 3D backbones to enhance the effectiveness of 3D learning, as point clouds are often accompanied by high-resolution multiview images provided by 3D capture devices.

References

[1] Iro Armeni, Ozan Sener, Amir Roshan Zamir, Helen Jiang, Ioannis K. Brilakis, Martin Fischer, and Silvio Savarese. 3D semantic parsing of large-scale indoor spaces. In *CVPR*, 2016. 2, 8

[2] Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. BEiT: BERT pre-training of image transformers. In *ICLR*, 2022. 1, 2

[3] Gilad Baruch, Zhuoyuan Chen, Afshin Dehghan, Tal Dimry, Yuri Feigin, Peter Fu, Thomas Gebauer, Brandon Joffe, Daniel Kurz, Arik Schwartz, and Elad Shulman. ARKitScenes: A diverse real-world dataset For 3D indoor scene understanding using mobile RGB-D data. In *NeurIPS Workshop*, 2021. 8

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *NeurIPS*, 33:1877–1901, 2020. 1

[5] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. ShapeNet: An information-rich 3D model repository. arXiv:1512.03012 [cs.GR], 2015. 2, 11

[6] Qiang Chen, Qiman Wu, Jian Wang, Qinghao Hu, Tao Hu, Errui Ding, Jian Cheng, and Jingdong Wang. MixFormer: Mixing features across windows and dimensions. In *CVPR*, pages 5249–5259, 2022. 2

[7] Yukang Chen, Jianhui Liu, Xiaojuan Qi, Xiangyu Zhang, Jian Sun, and Jiaya Jia. LargeKernel3D: Scaling up kernels in 3D CNNs, 2023. 9

[8] Jaesung Choe, Chunghyun Park, Francois Rameau, Jaesik Park, and In So Kweon. PointMixer: MLP-Mixer for point cloud understanding. In *ECCV*, 2022. 9

[9] Christopher Choy, JunYoung Gwak, and Silvio Savarese. 4D spatio-temporal ConvNets: Minkowski convolutional neural networks. In *CVPR*, 2019. 8

[10] Xiangxiang Chu, Zhi Tian, Yuqing Wang, Bo Zhang, Haibing Ren, Xiaolin Wei, Huaxia Xia, and Chunhua Shen. Twins: Revisiting the design of spatial attention in vision transformers. *NeurIPS*, 34:9355–9366, 2021. 2

[11] Angela Dai, Angel X. Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. ScanNet: Richly-annotated 3D reconstructions of indoor scenes. In *CVPR*, 2017. 1, 2, 6, 8

[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019. 1

[13] Runpei Dong, Zekun Qi, Linfeng Zhang, Junbo Zhang, Jianjian Sun, Zheng Ge, Li Yi, and Kaisheng Ma. Autoencoders as cross-modal teachers: Can pretrained 2D image transformers help 3D representation learning? In *ICLR*, 2023. 2

[14] Xiaoyi Dong, Jianmin Bao, Dongdong Chen, Weiming Zhang, Nenghai Yu, Lu Yuan, Dong Chen, and Baining Guo. CSWin transformer: A general vision transformer backbone with cross-shaped windows. In *CVPR*, pages 12124–12134, 2022. 2

[15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. 1, 2

[16] Meng-Hao Guo, Jun-Xiong Cai, Zheng-Ning Liu, Tai-Jiang Mu, Ralph R Martin, and Shi-Min Hu. PCT: Point cloud transformer. *Computational Visual Media*, 7(2):187–199, 2021. 2

[17] Meng-Hao Guo, Tian-Xing Xu, Jiang-Jiang Liu, Zheng-Ning Liu, Peng-Tao Jiang, Tai-Jiang Mu, Song-Hai Zhang, Ralph R Martin, Ming-Ming Cheng, and Shi-Min Hu. Attention mechanisms in computer vision: A survey. *Computational Visual Media*, pages 1–38, 2022. 2

[18] JunYoung Gwak, Christopher Choy, and Silvio Savarese. Generative sparse detection networks for 3D single-shot object detection. In *ECCV*, pages 297–313, 2020. 10

[19] JunYoung Gwak, Christopher B Choy, and Silvio Savarese. Generative sparse detection networks for 3D single-shot object detection. In *ECCV*, 2020. 11

[20] Kai Han, Yunhe Wang, Hanting Chen, Xinghao Chen, Jianyuan Guo, Zhenhua Liu, Yehui Tang, An Xiao, Chunjing Xu, Yixing Xu, et al. A survey on vision transformer. *IEEE Trans. Pattern Anal. Mach. Intell.*, 2022. 2

[21] Qi Han, Zejia Fan, Qi Dai, Lei Sun, Ming-Ming Cheng, Jiaying Liu, and Jingdong Wang. Demystifying local vision transformer: Sparse connectivity, weight sharing, and dynamic weight. In *ICLR*, 2022. 2

[22] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross B. Girshick. Masked autoencoders are scalable vision learners. In *CVPR*, pages 15979–15988, 2022. 2

[23] Ji Hou, Benjamin Graham, Matthias Nießner, and Saining Xie. Exploring data-efficient 3D scene understanding with contrastive scene contexts. In *CVPR*, 2021. 2, 8, 9

[24] Tianyu Huang, Bowen Dong, Yunhan Yang, Xiaoshui Huang, Rynson WH Lau, Wanli Ouyang, and Wangmeng Zuo. Clip2Point: Transfer clip to point cloud classification with image-depth pre-training. arXiv:2210.01055, 2022. 2

[25] Xiaoshui Huang, Sheng Li, Wentao Qu, Tong He, Yifan Zuo, and Wanli Ouyang. Frozen CLIP model is efficient point cloud backbone. arXiv:2212.04098, 2022. 2

[26] Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. Transformers in vision: A survey. *ACM Computing Surveys (CSUR)*, 2021. 2

[27] Jean Lahoud, Jiale Cao, Fahad Shahbaz Khan, Hisham Cholakkal, Rao Muhammad Anwer, Salman Khan, and Ming-Hsuan Yang. 3D vision with transformers: A survey, 2022. 2

[28] Xin Lai, Jianhui Liu, Li Jiang, Liwei Wang, Hengshuang Zhao, Shu Liu, Xiaojuan Qi, and Jiaya Jia. Stratified transformer for 3D point cloud segmentation. In *CVPR*, pages 8500–8509, 2022. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 15

[29] Wei Li, Xing Wang, Xin Xia, Jie Wu, Xuefeng Xiao, Min Zheng, and Shiping Wen. SepViT: Separable vision transformer. arXiv:2203.15380[cs.CV], 2022. 2

[30] Haojia Lin, Xiawu Zheng, Lijiang Li, Fei Chao, Shanshan Wang, Yan Wang, Yonghong Tian, and Rongrong Ji. Meta architecture for point cloud analysis. In *CVPR*, 2023. 9

[31] Haotian Liu, Mu Cai, and Yong Jae Lee. Masked discrimination for self-supervised learning on point clouds. In *ECCV*, 2022. 2

[32] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, et al. Swin transformer V2: Scaling up capacity and resolution. In *CVPR*, pages 12009–12019, 2022. 1

[33] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, pages 10012–10022, 2021. 1, 2

[34] Alexey Nekrasov, Jonas Schult, Or Litany, Bastian Leibe, and Francis Engelmann. Mix3D: Out-of-context data augmentation for 3D scenes. In *3DV*, pages 116–125, 2021. 9

[35] Yatian Pang, Wenxiao Wang, Francis E. H. Tay, W. Liu, Yonghong Tian, and Liuliang Yuan. Masked autoencoders for point cloud self-supervised learning. In *ECCV*, 2022. 2

[36] Chunghyun Park, Yoonwoo Jeong, Minsu Cho, and Jaesik Park. Fast point transformer. In *CVPR*, pages 16949–16958, 2022. 2, 9

[37] Guocheng Qian, Yuchen Li, Houwen Peng, Jinjie Mai, Hasan Abed Al Kader Hammoud, Mohamed Elhoseiny, and Bernard Ghanem. PointNeXt: Revisiting PointNet++ with improved training and scaling strategies. In *NeurIPS*, 2022. 9

[38] Haoxi Ran, Jun Liu, and Chengjie Wang. Surface representation for point clouds. In *CVPR*, pages 18942–18952, 2022. 10

[39] Yongming Rao, Benlin Liu, Yi Wei, Jiwen Lu, Cho-Jui Hsieh, and Jie Zhou. RandomRooms: Unsupervised pre-training from synthetic shapes and randomized layouts for 3D object detection. In *ICCV*, pages 3283–3292, 2021. 10, 11

[40] Danila Rukhovich, Anna Vorontsova, and Anton Konushin. FCAF3D: Fully convolutional anchor-free 3D object detection. In *ECCV*, 2021. 8, 10, 11

[41] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *NAACL*, 2018. 5

[42] Hugues Thomas, Charles R. Qi, Jean-Emmanuel Deschaud, Beatriz Marcotegui, François Goulette, and Leonidas J. Guibas. KPConv: Flexible and deformable convolution for point clouds. In *ICCV*, 2019. 4

[43] Zhengzhong Tu, Hossein Talebi, Han Zhang, Feng Yang, Peyman Milanfar, Alan Bovik, and Yinxiao Li. MaxViT: Multi-axis vision transformer. In *ECCV*, 2022. 2

[44] Thang Vu, Kookhoi Kim, Tung M Luu, Thanh Nguyen, and Chang D Yoo. SoftGroup for 3D instance segmentation on point clouds. In *CVPR*, pages 2708–2717, 2022. 10

[45] Haiyang Wang, Lihe Ding, Shaocong Dong, Shaoshuai Shi, Aoxue Li, Jianan Li, Zhenguo Li, and Liwei Wang. CA-Group3D: Class-aware grouping for 3D object detection on point clouds. In *NeurIPS*, 2022. 8, 10, 11

[46] Peng-Shuai Wang. OctFormer: Octree-based transformers for 3D point clouds . *ACM Trans. Graph.*, 2023. 8, 9

[47] Peng-Shuai Wang, Yang Liu, Yu-Xiao Guo, Chun-Yu Sun, and Xin Tong. O-CNN: Octree-based convolutional neural networks for 3D shape analysis. *ACM Trans. Graph.*, 36(4):72:1–72:11, 2017. 8, 9

[48] Peng-Shuai Wang, Yu-Qi Yang, Qian-Fang Zou, Zhirong Wu, Yang Liu, and Xin Tong. Unsupervised 3D learning for shape analysis via multiresolution instance discrimination. In *AAAI*, 2020. 2

[49] Qi Wang, Sheng Shi, Jiahui Li, Wuming Jiang, and Xiangde Zhang. Window normalization: Enhancing point cloud understanding by unifying inconsistent point densities. arXiv:2212.02287, 2022. 9

[50] Wenxiao Wang, Lu Yao, Long Chen, Binbin Lin, Deng Cai, Xiaofei He, and Wei Liu. Crossformer: A versatile vision transformer hinging on cross-scale attention. In *ICLR*, 2022. 2

[51] Ziyi Wang, Xumin Yu, Yongming Rao, Jie Zhou, and Jiwen Lu. P2P: Tuning pre-trained image models for point cloud analysis with point-to-pixel prompting. In *NeurIPS*, 2022. 2

[52] Kan Wu, Houwen Peng, Minghao Chen, Jianlong Fu, and Hongyang Chao. Rethinking and improving relative position encoding for vision transformer. In *ICCV*, pages 10033–10041, 2021. 1, 2, 5

[53] Sitong Wu, Tianyi Wu, Haoru Tan, and Guodong Guo. Pale transformer: A general vision transformer backbone with pale-shaped attention. In *AAAI*, pages 2731–2739, 2022. 2

[54] Wenxuan Wu, Qi Shan, and Li Fuxin. PointConvFormer: Revenge of the point-based convolution. In *CVPR*, 2023. 9

[55] Xiaoyang Wu, Yixing Lao, Li Jiang, Xihui Liu, and Hengshuang Zhao. Point Transformer V2: Grouped vector attention and partition-based pooling. In *NeurIPS*, 2022. 2, 8, 9, 10

[56] Xiaoyang Wu, Xin Wen, Xihui Liu, and Hengshuang Zhao. Masked scene contrast: A scalable framework for unsupervised 3D representation learning. In *CVPR*, pages 9415–9424, 2023. 2, 8, 9, 12

[57] Saining Xie, Jiatao Gu, Demi Guo, Charles R Qi, Leonidas J Guibas, and Or Litany. PointContrast: Unsupervised pre-training for 3D point cloud understanding. *ECCV*, 2020. 2, 8, 9, 10, 11

[58] Jianwei Yang, Chunyuan Li, Pengchuan Zhang, Xiyang Dai, Bin Xiao, Lu Yuan, and Jianfeng Gao. Focal self-attention for local-global interactions in vision transformers. *NeurIPS*, 34:30008–30022, 2021. 2

[59] Xumin Yu, Lulu Tang, Yongming Rao, Tiejun Huang, Jie Zhou, and Jiwen Lu. Point-BERT: Pre-training 3D point cloud transformers with masked point modeling. In *CVPR*, pages 19313–19322, June 2022. 2

[60] Yuhui Yuan, Rao Fu, Lang Huang, Weihong Lin, Chao Zhang, Xilin Chen, and Jingdong Wang. HRFormer: High-resolution vision transformer for dense prediction. *NeurIPS*, 34:7281–7293, 2021. 2

[61] Qiming Zhang, Yufei Xu, Jing Zhang, and Dacheng Tao. VSA: Learning varied-size window attention in vision transformers. In *ECCV*, 2022. 2

[62] Renrui Zhang, Ziyu Guo, Peng Gao, Rongyao Fang, Bin Zhao, Dong Wang, Yu Qiao, and Hongsheng Li. Point-M2AE: Multi-scale masked autoencoders for hierarchical point cloud pre-training. In *NeurIPS*, 2022. 2, 10, 11

[63] Zaiwei Zhang, Rohit Girdhar, Armand Joulin, and Ishan Misra. Self-supervised pretraining of 3D features on any point-cloud. In *ICCV*, pages 10252–10263, 2021. 2, 8, 9

[64] Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip HS Torr, and Vladlen Koltun. Point transformer. In *ICCV*, pages 16259–16268, 2021. 2

[65] Jia Zheng, Junfei Zhang, Jing Li, Rui Tang, Shenghua Gao, and Zihan Zhou. Structured3D: A large photo-realistic dataset for structured 3D modeling. In *ECCV*, pages 519–535, 2020. 1, 6

[66] Chuhang Zou, Jheng-Wei Su, Chi-Han Peng, Alex Colburn, Qi Shan, Peter Wonka, Hung-Kuo Chu, and Derek Hoiem. Manhattan room layout reconstruction from a single 360 image: A Comparative study of state-of-the-art methods. *International Journal of Computer Vision*, 129:1410–1431, 2021. 6

Appendix: Efficient self-attention implementation

Apart from our memory-efficient self-attention approach, we have also enhanced our self-attention computation in the following ways.

Kernel scheduling Designing an optimal scheduling strategy for CUDA kernels is essential to make the most of the memory bandwidth and computational capabilities of modern GPUs. Stratified Transformer [28] builds CUDA blocks whose number is equal to the number of points in the window and creates GPU threads whose number is the maximum number of points in the window. However, the varying number of points in each window makes it difficult for the CUDA compiler to optimize the execution speed. To tackle this issue, our kernel is designed to calculate the channel-wise contribution to the weight coefficients $c_{ij,h,d}$ (i, j are the indices of queries and keys, h is the index of the head, d is the index of the channel) per thread and binds blocks to a fixed number of threads (256 in our implementation). We use a local synchronized operation (*__shfl_down_sync*) to perform *reduce-sum* to obtain the coefficient weight $c_{ij,h}$. Thanks to the compacted memory access between neighboring threads and fixed and balanced operations per thread, our implementation results in higher memory bandwidth and efficient computation.

Half-precision support We enable half-precision for all CUDA kernels that we have implemented. Additionally, we have reorganized the memory layout of look-up tables from dimension-last to channel-last. This allows a single thread to process two consecutive elements with a single 32-bit memory access for the query, key, and look-up tables, respectively.

Computation speedup We combine the calculation of coefficient weights and cRPE/cRSE into a single kernel, thus decreasing the memory access of queries and keys and speeding up GPU performance.

Reduction of atomic operations We employ atomic operations to compute the updated features and gradients of the lookup tables. However, half-precision atomic operators can be inefficient due to double writing/reading conflicts. To address this issue, we use shared memory to combine two consecutive 16-bit atomic operations into a single 32-bit atomic operation, thus reducing the number of atomic operations and speeding up GPU execution.